Practical Work (TP 1)

Introduction to Information Retrieval & Search Engine Architecture

Course	Information Web Retrieval and Analytics
Level	Engineering
Topic	Basics of IR, differences with databases, and search engine architecture

Name:	LY Chungheang	**Student ID:**	e20220980
Group:	I5_AMS_A	**Date:**	14/05/2026

Instructions

- Answer all **5 questions** in this document, in your own words.
- Submit **one PDF** by email (my email: dorn.dawin@gmail.com) and named TP1_StudentID_Name.pdf by **[14/05/2026 : 11:59PM]**.
- Work individually. If you use an AI tool, declare it at the end and copy or screenshot your prompt.

Question	Topic	Marks
1	Information Retrieval vs. Relational Databases	20
2	Text Transformation Pipeline	20
3	Search Engine Architecture & Processes	20
4	Inverted Index	20
5	Basic Scoring Model	20
	Total	**100**

Question 1 · Information Retrieval vs. Relational Databases (20 pts)

Compare Information Retrieval (IR) systems with traditional Relational Database Management Systems (RDBMS) based on the following criteria. Give an example for each criterion.

Criterion	Relational database (RDBMS)	Information retrieval (IR) system
1. Data type / structure	Structured records stored in tables with well-defined fields (attributes) and a fixed schema. For Example: bank records with account number, balance, name, address and date of birth.	Unstructured or semi-structured free text (web pages, e-mail, books, news stories, PDFs). It has some structure (title, author, date) but no fixed meaning per field. For Example: a collection of news stories or web pages.
2. Querying & matching strategy	Formal and precise query (SQL) with exact matching: a row matches or it does not. For Example: SELECT balance FROM accounts WHERE account_no= 123.	A short, vague query of a few words, matched partially by degree of relevance, because the same idea can be written with different words (fees vs charges, increase vs rise). For Example: the query "bank fees increase".
3. Output / result format	A set of rows that satisfy the condition, with no ranking. For Example: the single balance value of account 980.	A ranked list of documents ordered by degree of relevance, usually shown with titles and snippets. For Example: the ranked results for "cheap flight Phnom Penh Paris".

Question 2 · Text Transformation Pipeline (20 pts)

During the **Text Transformation** phase of the indexing process, text undergoes several preprocessing steps. Trace the following raw HTML string through each step of the pipeline:

Raw input:<h1>Library News</h1> The students are searching the libraries for useful books.

Use these rules:

- **Tokenizer:** convert to lower case and remove punctuation.
- **Stop list:** a, an, and, are, as, at, be, by, for, from, in, is, it, of, on, or, the, to, was, were, with.
- **Stemmer:** the Porter stemmer. You can check stems with an online Porter stemmer or with Python: from nltk.stem import PorterStemmer.

List the **explicit list of terms** output at each stage, and explain in one or two sentences what the step did.

Stage	Output (list of terms)	Explanation
1. Parser & Tokenizer identify structural markup, remove tags, produce tokens	[library, news, the, students, are, searching, the, libraries, for, useful, books]	The parser recognized the HTML markup (<h1>, <p>) and removed the tags, keeping only the text. The tokenizer then lower-cased it, removed the punctuation (the final full stop) and split it into 11 tokens.
2. Stop word removal remove common functional words	[library, news, students, searching, libraries, useful, books]	Removed the stop-list words "the" (twice), "are" and "for": frequent functional words that carry little meaning for searching. The list shrinks from 11 to 7 terms.
3. Stemming reduce words to their root or stem forms	[librari, news, student, search, librari, use, book]	The Porter stemmer strips suffixes (-ies, -s, -ing, -ful) so that word variants share one stem. "library" and "libraries" both become "librari", so a query on one matches the other. (Result of NLTK's default PorterStemmer; stems need not be real words.)

Question 3 · Search Engine Architecture & Processes (20 pts)

A modern search engine architecture is divided into two primary sub-processes: the **Indexing Process** and the **Query Process**.

3.1 Name and explain the core responsibilities of the **three components in the Indexing Process**. (7 pts)

Component	Core responsibilities
Text acquisition	Identifies and stores the documents for indexing. A crawler finds and acquires documents (web crawlers follow links; coverage and freshness matter). Feeds (e.g. RSS) supply real-time streams of new documents. Conversion turns HTML, Word, PDF, etc. into consistent text plus metadata (UTF-8). The document data store keeps text, metadata, links and anchor text for fast access (e.g. snippets).
Text transformation	Transforms documents into index terms or features. The parser recognises structure (titles, headings, links) from the markup; the tokenizer recognises words (capitalization, punctuation); then stop word removal and stemming. Link analysis (e.g. PageRank, anchor text), information extraction (named entities) and classifiers (topic, sentiment, spam) can add further features.
Index creation	Takes the index terms and builds data structures for fast searching: document statistics (counts and positions of words), weighting (e.g. tf·idf), inversion (document → terms into term → documents, the inverted index, with compression and updates) and index distribution across machines (document or term distribution, replication).

3.2 Name and explain the core responsibilities of the **three components in the Query Process**.

(7 pts)

Component	Core responsibilities
User interaction	Supports creating and refining the query and displaying the results. Query input gives the interface and query language; query transformation improves the query (spell checking and suggestion, query expansion such as car → car OR automobile, relevance feedback); results output builds the ranked display with snippets and highlighted words.
Ranking	Uses the query and the indexes to generate a ranked list of documents. Scoring applies the ranking algorithm (basic form $\Sigma\, q_i d_i$), the core of the search engine; performance optimization keeps it efficient (term-at-a-time vs document-at-a-time); distribution uses a query broker to send queries to machines and assemble the results, with caching.
Evaluation	Monitors and measures effectiveness and efficiency. Logging of queries and clickthrough data (online) supports query suggestion, spell checking, caching and ranking; ranking analysis and performance analysis (offline, on logs and test collections) measure and tune ranking effectiveness and system efficiency.

3.3 Discuss the fundamental trade-off search engine architects face between **Effectiveness** and **Efficiency**. (6 pts)

The architecture of a search engine is determined by two requirements. Effectiveness is the quality of the results (precision and recall). Efficiency is response time and throughput (latency, queries per second). They pull against each other: richer ranking functions, more features, query expansion and positional information improve quality but need more computation and storage, so queries become slower and costlier. Shortcuts such as simpler scoring, index compression, caching and unsafe optimizations save time and resources but can lower result quality. Users expect answers in a fraction of a second at huge scale, so designing a search engine is a constant trade-off between the two, for example by using a cheap function to select candidates and an expensive one to re-rank only the top results.

Question 4 · Inverted Index (20 pts)

Consider the following collection of four documents:

Doc	Text
D1	web search engines rank web pages
D2	search engines index documents
D3	users search the web for documents
D4	engines crawl pages and index pages

4.1 Explain what **term inversion** means in index creation: moving from **Document → Terms** to **Term → Documents**. (5 pts)

Documents are naturally stored as Document → Terms: each document lists the words it contains (D1 → web, search, engines, ...). That is poor for searching, because answering a query would mean scanning every document. Term inversion, the core of indexing, converts this into Term → Documents (the inverted index): the indexer reads each document, records every term with the documents it occurs in (and how often), and groups the entries by term, e.g. web → D1, D3. A query term then leads straight to its postings list without

scanning the collection, which gives a format designed for fast query processing. It is difficult for very large collections, and the index must also handle updates and use compression for efficiency.

4.2 Build the **inverted index** for this collection. Remove the stop words from the Question 2 stop list, but do **not** stem. List the terms in alphabetical order and, for each term, its postings with the term frequency, e.g. web → D1 (2), D3 (1). (10 pts)

Term	Postings: document (term frequency)
crawl	D4 (1)
documents	D2 (1), D3 (1)
engines	D1 (1), D2 (1), D4 (1)
index	D2 (1), D4 (1)
pages	D1 (1), D4 (2)
rank	D1 (1)
search	D1 (1), D2 (1), D3 (1)
users	D3 (1)
web	D1 (2), D3 (1)

4.3 Using **only your inverted index** (not the documents), find the documents that match each Boolean query, and explain how you combined the postings lists. (5 pts)

Query	Matching documents	How you combined the postings
search AND engines	D1, D2	search → {D1, D2, D3}; engines → {D1, D2, D4}. AND = intersection: keep only documents present in both postings lists → {D1, D2}.
web AND documents	D3	web → {D1, D3}; documents → {D2, D3}. AND = intersection → {D3}.
index OR crawl	D2, D4	index → {D2, D4}; crawl → {D4}. OR = union: all documents in either list, without duplicates → {D2, D4}.

Question 5 · Basic Scoring Model (20 pts)

In a simple vector/weighted IR model, the relevance score between a query q and a document d for shared terms i is calculated using:

$$\text{Score}(q, d) = \Sigma_i \, q_i \cdot d_i$$

where q_i is the query term weight and d_i is the document term weight.

5.1 Given a query vector **q = [1, 2]** and two document vectors **d_1 = [3, 0]** and **d_2 = [1, 2]**, calculate Score(q, d_1) and Score(q, d_2). Which document will be ranked higher? Explain why. (8 pts)

	Calculation	Score
Score(q, d_1)	$(1 \times 3) + (2 \times 0) = 3 + 0$	3
Score(q, d_2)	$(1 \times 1) + (2 \times 2) = 1 + 4$	5

Using the basic score $\Sigma \, q_i d_i$, d_2 is ranked higher (5 > 3). The query gives the second term twice the weight of the first. d_1 only has weight on the first term (0 on the second), so it earns just a small contribution, whereas d_2 also matches the heavily weighted second term. A high weight on a term the query cares about contributes more to the score than a high weight on a less important term.

5.2 The table gives term weights for three documents over the vocabulary **[search, engine, web, index]**. The query is **q = [1, 1, 0, 2]**. Calculate the score of each document and give the final ranking. (12 pts)

	search	engine	web	index
q	1	1	0	2
d_1	3	0	2	0
d_2	1	2	0	1
d_3	0	1	1	3

Document	Calculation	Score	Rank
d_1	$(1\times3) + (1\times0) + (0\times2) + (2\times0) = 3 + 0 + 0 + 0$	3	3
d_2	$(1\times1) + (1\times2) + (0\times0) + (2\times1) = 1 + 2 + 0 + 2$	5	2
d_3	$(1\times0) + (1\times1) + (0\times1) + (2\times3) = 0 + 1 + 0 + 6$	7	1

Final ranking: d_3 (7) > d_2 (5) > d_1 (3). d_3 wins mainly because it has a large weight (3) on "index", the term the query weights most (2). d_1's weight on "web" is ignored because the query weight for "web" is 0.

Declaration of AI tool use

I used the following AI tools for this TP (write “none” if you used none), and for which questions: